



FOR JAVA DEVELOPERS MOVING TO NESTJS

Slow Handler. Everyone Waits.

Spring Boot to NestJS, episode 4: async and the event loop

In the normal NestJS request path, your handlers share one event-loop thread.

In Spring MVC, Tomcat gives each request a thread from its pool. While the pool has capacity, a handler that blocks occupies its own thread and the others carry on.

In NestJS, your handlers share one event-loop thread. The Node.js documentation says it precisely: a single JavaScript thread is used by default. Occupy it, and unrelated requests wait.

Versions on this sheet were current on 29 September 2026: Node 22, NestJS 12, Spring Boot 4.1.1.

In scope

- What one busy handler costs every other request
- Why await helps only when what it awaits yields
- A benchmark that measured itself, and how to avoid it
- What blocks the event loop, and the documented ways out
- Where Spring's @Async executor went

Not in scope

- Spring WebFlux and MVC async support (Callable, DeferredResult)
- Thread pool sizes: none is named, because none is needed here

The Spring handler sleeps and the NestJS one stays busy: not the same workload. What is compared is an unrelated request while one handler cannot progress.

01 The measurement

SPRING MVC, BLOCKING	slow 1513 ms, ping during it 4 ms
NESTJS, SYNCHRONOUS	slow 1500 ms, ping during it 1402 ms
NESTJS, AWAITING A TIMER	slow 1500 ms, ping during it 2 ms
ONE-PROCESS BENCHMARK	meant to send at 100 ms, sent at 1513 ms, timed 4 ms
PBKDF2SYNC	ping during it 1329 ms
PBKDF2 (ASYNC)	ping during it 1 ms, on Node's worker pool

The same 1.5 seconds. Three different costs to everyone else.



02 The handler that stops everyone

A handler that keeps the thread busy for 1.5 seconds. A ping sent 100 ms later waited for the rest of it.

```
nestjs-api/src/ep04-async/blocking.ts
```

```
@Get('slow')
slow() {
  const until = Date.now() + BLOCK_MS;
  while (Date.now() < until) {
    /* spin */
  }
  return { blockedMs: BLOCK_MS };
}
```

"a single JavaScript thread is used by default"

Node.js documentation: the event loop, "Don't Block the Event Loop", and worker_threads, checked 2026-09-29.

Ping during it: 1402 ms.

Run on a real machine, 2026-09-29, by scripts/verify-async.sh in the course repository.

03 The one change

The handler still takes 1.5 seconds. It awaits a real timer, suspends, and control returns to the event loop, so the ping runs.

```
nestjs-api/src/ep04-async/non-blocking.ts
```

```
@Get('slow')
async slow() {
  await new Promise((r) => setTimeout(r, WAIT_MS));
  return { waitedMs: WAIT_MS };
}
```

Awaiting a genuinely asynchronous operation lets the handler stay pending without occupying the event loop. Ping: 2 ms.

Run on a real machine, 2026-09-29, by scripts/verify-async.sh in the course repository.



04 What really blocks

Large JSON parsing and stringifying, complex regular expressions, long loops that never yield, and synchronous crypto, zlib and fs calls. Some Node functions offload their own work: asynchronous pbkdf2 runs on the worker pool.

```
nestjs-api/src/ep04-async/crypto.ts
```

```
@Get('hash-sync')
hashSync() {
  return { key: pbkdf2Sync('secret', 'salt', ITERATIONS, 64, 'sha512').toString('hex').slice(0, 8) };
}

@Get('hash-async')
async hashAsync() {
  const key = await pbkdf2Async('secret', 'salt', ITERATIONS, 64, 'sha512');
  return { key: key.toString('hex').slice(0, 8) };
}
```

"the Event Loop should orchestrate client requests, not fulfill them itself"

Node.js documentation: the event loop, "Don't Block the Event Loop", and worker_threads, checked 2026-09-29.

"Workers (threads) are useful for performing CPU-intensive JavaScript operations. They do not help much with I/O-intensive work."

Node.js documentation: the event loop, "Don't Block the Event Loop", and worker_threads, checked 2026-09-29.

05 The questions this answers

Why can one slow NestJS handler delay every other request?

When does await help, and when does it not?

What is the difference between Node's worker pool and worker_threads?

Where did Spring's @Async go?

Why can a benchmark in the same process hide the problem it measures?

Before you ship a NestJS handler

1. Keep event-loop work short
2. Await only helps when what it awaits yields
3. Use the async version of crypto, zlib and fs
4. Break long CPU work into pieces that yield, or use a worker thread
5. Put heavy jobs on a queue
6. Never time a server from inside its own process

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-async.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	Node.js: Don't Block the Event Loop https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-29 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.